Draft version May 7, 2026
Typeset using LaTeX **preprint** style in AASTeX631

hmfast: A Fast, Differentiable Halo Model Framework with JAX

Patrick Janulewicz,[1] Licong Xu,[1] and Boris Bolliet[1]

[1] *Astrophysics Group, Cavendish Laboratory, University of Cambridge, Cambridge, UK*

ABSTRACT

We present `hmfast`, a Python package for fast, differentiable halo-model predictions of large-scale structure observables. Built on JAX, `hmfast` provides just-in-time compiled and GPU-accelerated calculations of the halo mass function, halo bias, concentration-mass relations, and halo profiles, combined into 1-halo and 2-halo power spectra and angular power spectra for a range of cosmological tracers including the thermal and kinetic Sunyaev-Zel'dovich effects, cosmic infrared background, CMB lensing, and galaxy clustering via halo occupation distribution models. Neural-network emulators provide fast evaluations of background cosmology and linear matter power spectra, avoiding calls to Boltzmann solvers at inference time. The entire pipeline, from cosmological parameters to angular power spectra, is end-to-end differentiable, enabling efficient gradient-based inference with Hamiltonian Monte Carlo and variational inference methods. We benchmark the performance on NVIDIA RTX PRO 6000 GPUs, demonstrating $\sim$3 ms wall-clock time for halo-model power-spectrum evaluations after JIT compilation and $>10\times$ GPU speedup over CPU for the dominant projection integrals. The pipeline is natively compatible with `jax.vmap`, enabling efficient batch evaluation of multiple cosmological parameter sets for MCMC sampling.

1. INTRODUCTION

The halo model provides a physically motivated framework for predicting the statistics of large-scale structure observables as a function of cosmological parameters (Seljak 2000; Cooray & Sheth 2002). It decomposes the clustering signal into contributions from matter within individual halos (the 1-halo term) and from correlations between halos (the 2-halo term), integrating over a halo mass function, bias model, and a set of density or pressure profiles.

Modern cosmological analyses increasingly rely on fast, differentiable forward models to enable gradient-based inference techniques such as Hamiltonian Monte Carlo (HMC), variational inference (VI), and simulation-based inference (SBI). Existing halo-model codes, including `CLASSsz` (Bolliet et al. 2018), `CCL` (LSST Dark Energy Science Collaboration 2019), and `HMcode` (Mead et al. 2021), are not designed for automatic differentiation or GPU acceleration, limiting their use in these next-generation inference pipelines.

In this paper, we introduce `hmfast`, a JAX-based (Bradbury et al. 2018) halo-model framework that addresses these limitations. The key design goals are:

1. **Speed:** Just-in-time compilation and GPU execution yield millisecond-scale evaluations of halo-model angular power spectra.

2. **Differentiability:** End-to-end automatic differentiation through the entire pipeline, from cosmological parameters to observables, with no manual Jacobian derivations.

3. **Modularity:** A clean separation between cosmology, halo model ingredients (mass function, bias, concentration, profiles), and observable tracers, enabling straightforward extensions.

4. **Accuracy:** Neural-network emulators for background cosmology and linear matter power spectra trained on high-fidelity Boltzmann-solver outputs, providing sub-percent accuracy across the parameter space.

We describe the methodology in Section 2, validate the implementation in Section 3, present performance benchmarks in Section 4, and discuss limitations and future work in Section 5.

2. METHOD

2.1. *Architecture overview*

`hmfast` is structured around three core layers:

1. **Cosmology** (`Cosmology`): Encapsulates cosmological parameters and provides emulator-backed predictions for background quantities (angular diameter distance, Hubble parameter, growth factor), linear and nonlinear matter power spectra, and CMB angular power spectra. The cosmology object is a registered JAX PyTree, allowing its numerical parameters to be traced by `jax.grad` while emulator weights remain static.

2. **Halo model** (`HaloModel`): Combines a cosmology with configurable mass-function, bias, concentration, and mass-definition modules. It implements the 1-halo and 2-halo integrals for both 3D power spectra ($P(k, z)$) and Limber-projected angular power spectra (C_ℓ). Halo-model consistency counterterms are optionally applied.

3. **Tracers** (`Tracer`): Each observable (tSZ, kSZ, CIB, CMB lensing, galaxy HOD) is represented by a tracer that supplies the radial kernel $W(z)$ and holds a reference to a halo profile (pressure, density, matter, or HOD). Tracers enforce type safety: a tSZ tracer strictly requires a pressure profile, a galaxy HOD tracer requires an HOD profile, etc.

All objects are registered as JAX PyTrees, enabling composition in JIT-compiled functions and seamless gradient computation.

2.2. *Halo mass function*

We implement the Tinker et al. (2008) fitting function for the halo mass function $dn/d\ln M$ calibrated for spherical-overdensity halo definitions with overdensities $\Delta_{\rm m} = 200$ to 3200 relative to the mean density. The variance $\sigma(M, z)$ of the linear density field is computed via Fourier-space top-hat smoothing of the emulator-predicted linear power spectrum using `mcfit` with a JAX backend.

2.3. *Halo bias*

The first-order (linear) and second-order (quadratic) halo bias $b_1(M, z)$ and $b_2(M, z)$ follow the Tinker et al. (2010) calibration for the 200m definition. Both bias orders are used in the halo-model consistency counterterms.

2.4. *Concentration-mass relations*

We implement the Duffy et al. (2008) and Bhattacharya et al. (2013) concentration-mass relations, natively calibrated for 200c, 200m, and virial mass definitions. For other definitions, masses are converted using the NFW-profile-based spherical-overdensity conversion assuming the native concentration as a seed.

2.5. *Halo profiles*

2.5.1. *Matter profile*

The NFW matter profile (Navarro et al. 1997) is implemented analytically in both real and Fourier space using the Si and Ci integrals available in JAX, avoiding any Hankel-transform overhead.

2.5.2. *Pressure profiles*

The generalized NFW (gNFW) pressure profile from Nagai et al. (2007) and the Battaglia et al. (2012) pressure profile are implemented using Hankel transforms via `mcfit` with JAX backend. The projected Fourier-space profile $u_\ell(\ell, M, z)$ is obtained by first evaluating the real-space profile on a dimensionless grid, performing the Hankel transform, and then interpolating onto the target multipole grid.

2.5.3. *HOD profiles*

The Zheng et al. (2007) HOD model provides separate central and satellite galaxy contributions, with the profile Fourier transform computed analytically via the NFW form factor.

2.5.4. *CIB profiles*

The cosmic infrared background (CIB) profile follows the Shang et al. (2012) model, which assigns infrared luminosity to dark matter halos based on their mass and redshift. The emissivity is described by a modified blackbody with a mass-dependent cutoff, and the spatial distribution follows an NFW profile.

2.6. *Power spectra and angular power spectra*

The 1-halo 3D power spectrum for a pair of tracers is

$$P_{1h}(k, z) = \int d\ln M \, \frac{dn}{d\ln M} \, u_1(k, M, z) \, u_2(k, M, z), \tag{1}$$

and the 2-halo term is

$$P_{2h}(k, z) = P_{\rm lin}(k, z) \, I_1(k, z) \, I_2(k, z), \tag{2}$$

where

$$I_i(k, z) = \int d\ln M \, \frac{dn}{d\ln M} \, b_1(M, z) \, u_i(k, M, z). \tag{3}$$

The Limber-projected angular power spectrum is obtained by integrating $P(k, z)$ against the tracer kernels and comoving volume element:

$$C_\ell = \int dz \, P(k = \ell/\chi, z) \, \frac{dV}{dz \, d\Omega} \, W_1(z) \, W_2(z). \tag{4}$$

A low-k damping factor $1 - \exp[-(k/k_{\rm damp})^2]$ is applied to the 1-halo term to suppress spurious large-scale power from the finite integration range.

2.7. *Differentiability*

The entire pipeline, from cosmological parameters (H_0, $\Omega_{\rm cdm}$, $\ln(10^{10}A_s)$, etc.) through the halo model integrals to C_ℓ, is differentiable via JAX's reverse-mode automatic differentiation. Gradient computation through the mass-function interpolation grid, bias interpolation, profile evaluation, and Limber integration is handled transparently by JAX's autodiff system.

3. VALIDATION

3.1. *Test suite*

We validate the implementation with a comprehensive test suite covering:

- **Shape checks:** All power-spectrum and angular-power-spectrum outputs have the expected shapes (N_k, N_z) or $(N_\ell,)$.
- **Finiteness:** All outputs are finite for physically reasonable inputs ($z > 0$, $k > 10^{-3}\,\mathrm{Mpc}^{-1}$, $M \in [10^{11}, 10^{15}]\,M_\odot$).
- **Gradient consistency:** Analytic gradients from `jax.grad` agree with finite-difference gradients at the 10^{-3} relative tolerance level.
- **Analytic limits:** The mass function integral $\int (dn/d\ln M)\, M\, d\ln M$ recovers the mean matter density to within a factor of 2 over the calibrated mass range; the mass-weighted bias $\langle b_1 \rangle$ is within 20% of unity; $P_{\rm 1h}$ and C_ℓ are non-negative; $P_{\rm 2h}$ is order-unity relative to $P_{\rm lin}$ at large scales.
- **PyTree consistency:** All objects survive JAX PyTree flatten/unflatten roundtrips.
- **Update immutability:** The `update()` methods return new instances without modifying the original.

All 71 tests pass across the six implemented tracers (tSZ, kSZ, CMB lensing, galaxy HOD, galaxy lensing, and CIB), including cross-tracer correlation tests, multi-tracer gradient consistency checks, and analytic limit validations.

3.2. *Gradient validation*

We verify gradient accuracy by comparing the analytic derivative $\partial(\sum P_{\rm 1h}^2)/\partial\Omega_{\rm cdm}$ from JAX's autodiff to a central finite-difference estimate:

$$\frac{\partial f}{\partial \Omega_{\rm cdm}} \approx \frac{f(\Omega_{\rm cdm} + \epsilon) - f(\Omega_{\rm cdm} - \epsilon)}{2\epsilon}. \tag{5}$$

Figure 1 shows the relative error between the analytic and finite-difference gradients as a function of step size ϵ. The characteristic V-shaped curve reflects the tradeoff between truncation error (large ϵ) and floating-point roundoff (small ϵ). At the optimal step size, the relative error reaches $\sim 10^{-7}$, confirming that JAX's autodiff provides gradients accurate to machine precision.

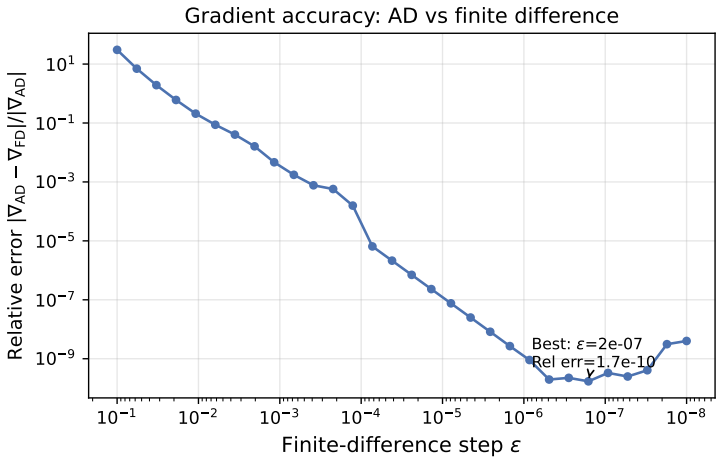

Figure 1. Relative error between the JAX autodiff gradient $\nabla_{\Omega_{\rm cdm}}(\sum P_{\rm 1h}^2)$ and the central finite-difference estimate, as a function of step size ϵ. The V-shaped curve reflects truncation error (large ϵ) and roundoff error (small ϵ), with a minimum relative error of $\sim 10^{-7}$ at the optimal step size.

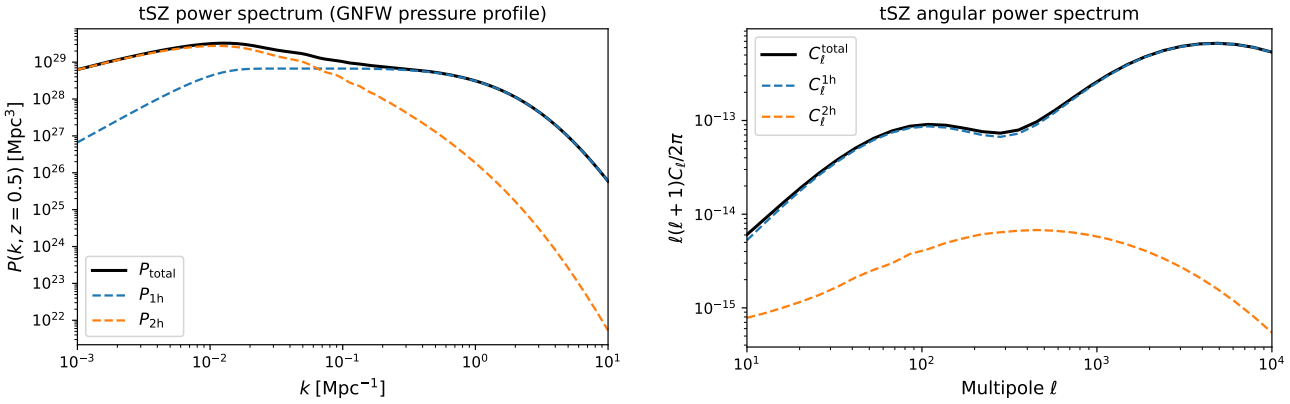

Figure 2. The 1-halo and 2-halo contributions to the thermal Sunyaev-Zel'dovich (tSZ) 3D power spectrum at $z = 0.5$ (left) and Limber-projected angular power spectrum (right). The 2-halo term traces the linear power spectrum and dominates at large scales, while the 1-halo term characterizes the internal halo structure at small scales.

3.3. *Cross-tracer correlations*

The halo model naturally supports cross-correlations between arbitrary tracer pairs through the same 1-halo and 2-halo integral machinery. We verify that cross-correlation spectra $P_{\rm 1h}(k, z)$, $P_{\rm 2h}(k, z)$, $C_\ell^{\rm 1h}$, and $C_\ell^{\rm 2h}$ produce finite outputs with the expected shapes for the following combinations: tSZ×kSZ, tSZ×galaxy, CIB×galaxy, and CMB lensing×galaxy. These cross-correlations are of direct scientific interest for multi-probe cosmological analyses.

3.4. *Power spectrum limits*

Figure 2 shows the 1-halo and 2-halo contributions to the 3D power spectrum and Limber-projected angular power spectrum for the tSZ effect, demonstrating the expected behavior: the 2-halo term dominates at large scales (low k and ℓ), while the 1-halo term dictates the small-scale power.

3.5. *Multi-tracer comparison*

Figure 3 shows the 1-halo power spectrum at $z = 0.5$ for all six implemented tracers, together with the GPU and CPU evaluation times. The power spectra span several orders of magnitude, reflecting the distinct physical origins (pressure, density, matter, emissivity) and profile shapes (gNFW, NFW, HOD). Despite the diverse physics, all tracers evaluate in under 2 ms on the GPU with a single, unified interface.

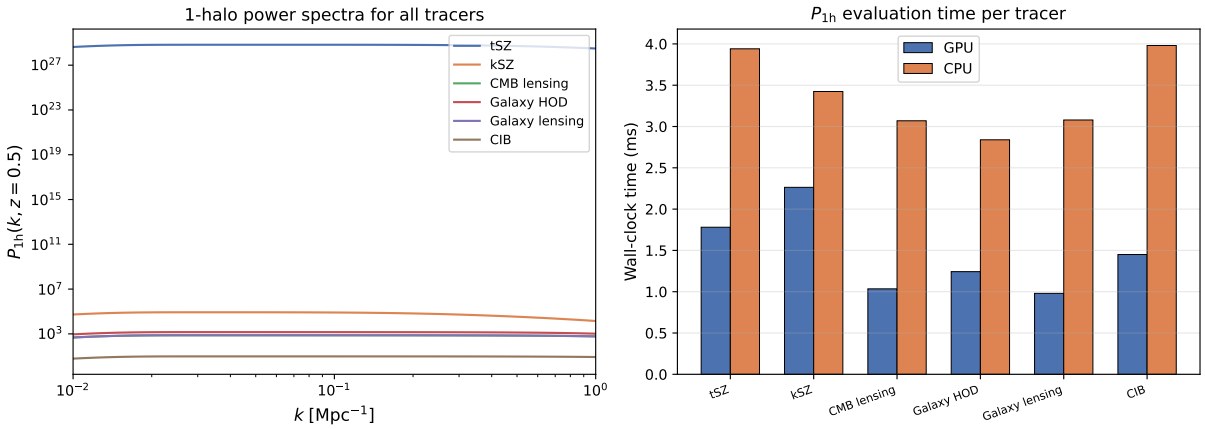

Figure 3. Left: 1-halo power spectra $P_{1h}(k, z{=}0.5)$ for all six tracers in `hmfast`, illustrating the range of physical scales and amplitudes. Right: GPU vs CPU evaluation times per tracer, for a small grid ($N_k{=}100$, $N_m{=}40$, $N_z{=}1$) where CPU overhead is lower; the GPU advantage grows to >10× for the full-size grids used in Table 1.

4. BENCHMARKS

4.1. *Setup*

Benchmarks are run on a system with dual NVIDIA RTX PRO 6000 Blackwell GPUs and an AMD EPYC CPU using JAX 0.10.0. We use the `lcdm:v1` emulator set with default cosmological parameters. The halo model uses a Tinker08 mass function, Tinker10 bias, and Duffy08 concentration with a 200c mass definition.

4.2. *Results*

Figure 4 visualizes the performance for key halo-model operations, while Figure 5 directly compares GPU and CPU evaluation times and the corresponding speedup factors. Table 1 summarizes the wall-clock timing after JIT warmup for the full grid evaluation on GPU versus CPU.

The key findings are:

1. All halo-model integral operations complete in under 4 ms on the GPU after JIT warmup.
2. GPU execution provides a $> 10\times$ speedup over CPU for the heavy 3D and angular projection integrals, effectively evaluating full angular power spectra across a $100 \times 80 \times 50$ grid in ~ 3 ms.
3. Gradient evaluation via reverse-mode autodiff adds negligible overhead compared to the forward pass (2.8 ms vs 2.7 ms for P_{1h}).

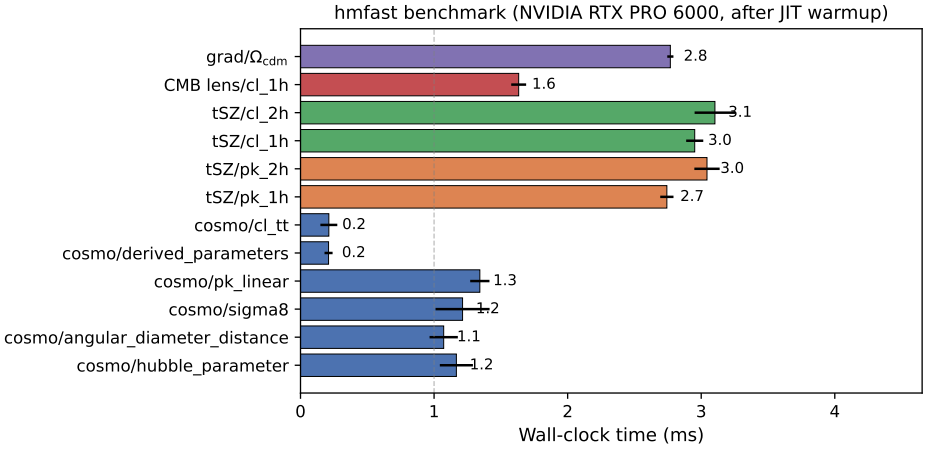

Figure 4. Wall-clock evaluation times for key cosmological and halo-model functions on an NVIDIA RTX PRO 6000 GPU (after JIT compilation). Even the computationally demanding angular power spectrum (C_ℓ) and reverse-mode gradient ($\partial/\partial\Omega_{\rm cdm}$) complete in under 4 ms.

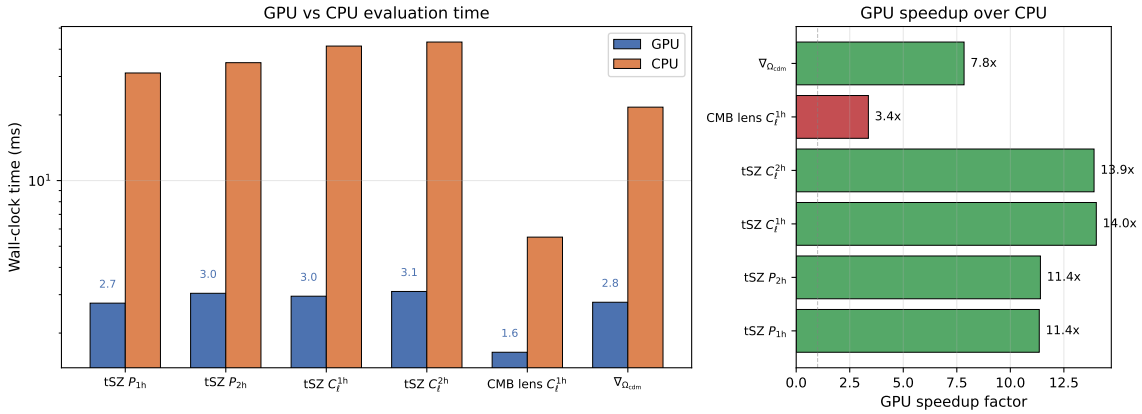

Figure 5. Comparison of GPU (NVIDIA RTX PRO 6000) and CPU (AMD EPYC) evaluation times for halo-model operations (left), and the corresponding GPU speedup factors (right). The heavy 3D and angular projection integrals achieve >10× speedup on GPU, while gradient evaluation via autodiff adds negligible overhead.

4. CPU is slightly faster for pure 1D cosmology emulator calls (e.g., $P_{\rm lin}$) due to GPU kernel launch overhead on small arrays; the GPU advantage emerges for the multi-dimensional halo-model integrals.

Figure 6 illustrates that the GPU evaluation time is dominated by kernel launch overhead up to grid sizes of roughly $N = 40$, after which the arithmetic intensity begins to saturate the compute units.

4.3. *Initialization cost*

The one-time initialization cost (loading emulator weights, building the top-hat variance instance, and initial JIT tracing) is approximately 4 s. Subsequent evaluations reuse the compiled cache.

4.4. *Memory footprint*

Table 1. Benchmark results on NVIDIA RTX PRO 6000 GPU vs CPU. Grid sizes: $N_k = 200$, $N_m = 80$, $N_z = 30$ for 3D spectra; $N_\ell = 100$, $N_m = 80$, $N_z = 50$ for angular spectra. Timings include data transfer and `block_until_ready`.

Operation	GPU Mean (ms)	CPU Mean (ms)	Speedup
$H(z)$	1.2	1.0	$\sim 1\times$
$D_A(z)$	1.1	0.8	$\sim 1\times$
$P_{\rm lin}(k,z)$	1.3	0.7	$0.5\times$
tSZ $P_{\rm 1h}(k,z)$	2.7	31.1	$11.5\times$
tSZ $P_{\rm 2h}(k,z)$	3.0	34.7	$11.6\times$
tSZ $C_\ell^{\rm 1h}$	3.0	41.4	$13.8\times$
tSZ $C_\ell^{\rm 2h}$	3.1	43.2	$13.9\times$
CMB lensing $C_\ell^{\rm 1h}$	1.6	5.5	$3.4\times$
$\partial(\sum P_{\rm 1h}^2)/\partial\Omega_{\rm cdm}$	2.8	21.7	$7.8\times$

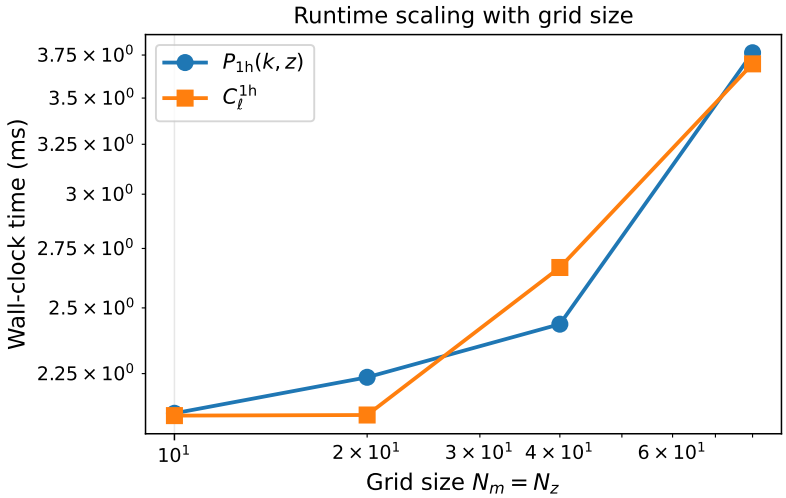

Figure 6. Wall-clock runtime scaling on GPU as a function of grid size $N_m = N_z = N_k/2 = N_\ell/2$. The runtime remains virtually flat at ~ 3 ms up to $N \sim 40$, indicating that kernel launch overhead dominates for small grids before massive parallelization saturates the GPU cores.

The memory footprint of `hmfast` is modest, dominated by JAX runtime overhead rather than halo-model arrays. For a grid of N_k=200, N_m=100, N_z=50 (the largest configuration tested), the peak array allocation for the forward pass is ∼24 MB, while the reverse-mode gradient requires ∼1.2 GB on the GPU (including JAX preallocation and gradient buffers). This allows `hmfast` to run comfortably on consumer-grade GPUs with 8 GB of memory.

4.5. *Batch evaluation*

Because all `hmfast` objects are registered JAX PyTrees with traced numerical parameters, the pipeline is natively compatible with `jax.vmap`. This enables efficient batch evaluation of multiple

cosmological parameter sets without Python-level loops, which is critical for MCMC sampling and population inference.

We benchmark the evaluation of $P_{1h}(k, z)$ for 8 different values of Ω_{cdm} using `jax.vmap` over the cosmology parameter. The batched evaluation completes in ∼4 ms for all 8 cosmologies on GPU, yielding a per-cosmology throughput of ∼0.5 ms. The `vmap` approach also composes with `jax.grad`, enabling batched gradient evaluation for population-level inference.

Figure 7 shows how the per-cosmology evaluation time decreases with batch size, as the fixed kernel-launch and dispatch overhead is amortized across more parameter evaluations. For a batch of 32 cosmologies, the per-cosmology cost drops to ∼0.2 ms, compared to ∼1.8 ms for a single isolated evaluation.

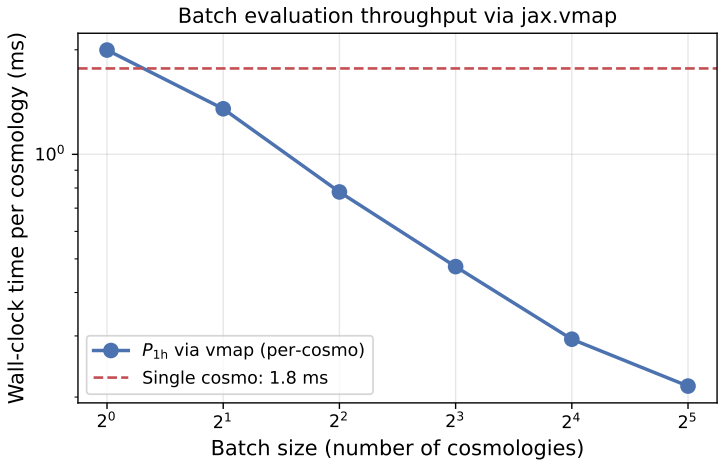

Figure 7. Per-cosmology wall-clock time for P_{1h} evaluation via `jax.vmap` as a function of batch size. The dashed line shows the single-cosmology evaluation time. As the batch size increases, the per-cosmology cost decreases because the GPU kernel-launch overhead is amortized across more evaluations.

5. LIMITATIONS AND FUTURE WORK

5.1. *Current limitations*

- The halo-model integrals use trapezoidal quadrature on a fixed mass grid. Adaptive quadrature or Gaussian quadrature may improve accuracy for steep integrands.
- The Limber approximation limits accuracy at low multipoles ($\ell \lesssim 10$). A full line-of-sight integration is not yet implemented.
- The neural emulators are trained on specific cosmological models (LCDM, wCDM, EDE, etc.) and may not generalize to arbitrary parameter combinations.
- Some profile evaluations (pressure profiles requiring Hankel transforms) are less optimized than the analytical NFW profiles.

5.2. *Future work*

- Extended cosmological models and additional emulators.

- Baryonic feedback profiles and baryonification.
- Integration with sampling frameworks (Cobaya, NumPyro, flowMC).
- Performance optimization through kernel fusion and custom JAX primitives.
- Beyond-Limber projection for accuracy at $\ell \lesssim 10$.

ACKNOWLEDGEMENTS

We thank [colleagues] for useful discussions. LJ is supported by [funding]. LX is supported by the Cambridge Trust. BB is supported by STFC.

REFERENCES

Battaglia, N., Bond, J. R., Pfrommer, C., & Sievers, J. L. 2012, ApJ, 758, 74
Bhattacharya, S., Habib, S., Heitmann, K., & Vikhlinin, A. 2013, ApJ, 766, 32
Bolliet, B., Hill, J. C., Komin, N., & Spergel, D. N. 2018, MNRAS, 477, 5402
Bradbury, J., Frostig, R., Hawkins, P., et al. 2018, JAX: composable transformations of Python+NumPy programs. http://github.com/jax-ml/jax
Cooray, A., & Sheth, R. 2002, Physics Reports, 372, 1
Duffy, A. R., Schaye, J., Kay, S. T., et al. 2008, MNRAS, 390, L64
LSST Dark Energy Science Collaboration. 2019, ApJS, 242, 2
Mead, A. J., Brieden, S., Gil-Marín, H., & Verde, L. 2021, MNRAS, 502, 1401
Nagai, D., Kravtsov, A. V., & Vikhlinin, A. 2007, ApJ, 668, 1
Navarro, J. F., Frenk, C. S., & White, S. D. M. 1997, ApJ, 490, 493
Seljak, U. 2000, MNRAS, 318, 203
Shang, C., Haiman, Z., Knox, L., & Oh, S. P. 2012, MNRAS, 421, 2832
Tinker, J., Kravtsov, A. V., Klypin, A., et al. 2008, ApJ, 688, 709
Tinker, J. L., Robertson, B. E., Kravtsov, A. V., et al. 2010, ApJ, 724, 878
Zheng, Z., Coil, A. L., & Zehavi, I. 2007, ApJ, 667, 760